

R_Markdown-Assignment

Kelly Clark

2026-04-27

Contents

Section 1: Create, Save, and Knit Your First R Markdown File	2
1.1 How I Created a New R Markdown File	2
1.2 Install and Load Packages	2
Section 2: Formatting Text Using RMarkdown Syntax	3
2.1. Headings	3
2.2 Italics	4
2.3 Bold	5
2.4 Bold and Italics	6
2.5 Bulleted list	6
2.6 Numbered List	7
Section 3: Inserting Code Chunks and Running Basic R Code	8
Part A: Create a Scatterplot	16
Part B: Create a Boxplot	18
Part C: Create a Summary Table	20
Part D: Compare the Visuals	21
Part E: Now you try	21
	22
Section 5: Improve and Finalize Your R Markdown Report	22
Step 1: Understanding Inline R Code	22
Step 2: Improving Readability by Hiding Code	23
Step 3: Removing Warnings and Messages	25
Step 4: Hiding Code AND Output with <code>include=FALSE</code>	26
Step 5: Controlling Figure Size	27
Step 6: Adding a Table of Contents	28

Step 7: Adding Hyperlinks	29
Step 8: Writing a Strong Conclusion	29
Step 9: Final Check and Knit	30

Section 1: Create, Save, and Knit Your First R Markdown File

Author: Jaslyn Davenport (jcdavenport@uncg.edu)

Objective: Learn how to create and generate a basic R Markdown file.

.....

1.1 How I Created a New R Markdown File

.....

To begin, open RStudio and create a new R Markdown file.

Go to **File > New File > R Markdown**.

QUESTION: What do you think happens when the new file is opened?

ANSWER: A new markdown file is created.

.....

1.2 Install and Load Packages

.....

Before knitting your file, make sure the required packages are installed and loaded.

TASK: Run the following in the Console:

```
install.packages("rmarkdown") install.packages("knitr") library(rmarkdown) library(knitr) install.packages("tinytex")
tinytex::install_tinytex()
```

QUESTION: Why are these packages needed? ANSWER:Because we wouldn't be able to do the assignment without it.

..... ## 1.3 Modify the YAML Header

At the top of the file, you will see the YAML header between two sets of — This section controls the document settings.

TASK: Change the YAML header so that it includes: a new title your name as the author the date PDF output

Example: title: "My First R Markdown File" author: "Jaslyn Davenport" date: "April 2026" output: pdf_document

QUESTION: What is the purpose of the YAML header? ANSWER: It's the metadata for the R markdown file. To tell the computer how to store this file and who created it and when.

..... ##1.4 Save and Knit file

After editing the file, save it as an .Rmd file. Then click the Knit button at the top of RStudio.

If you get an error message about LaTeX or TinyTeX, install TinyTeX by running: install.packages("tinytex") tinytex::install_tinytex() Then click Knit again.

QUESTION: What happens when you click Knit? ANSWER: It saves the R markdown file to my workign directory with the changes that I have made.

QUESTION: What type of file is created? ANSWER: A pdf.

..... ##1.5 Final observation After knitting, compare the .Rmd file to the final PDF.

QUESTION: How is the output file different from the source file? ANSWER: It looks so fancy. It looks like I wrote the report in word but saved it as a pdf.

Section 2: Formatting Text Using RMarkdown Syntax

Author: Arnav Pandey (a_pandey2@uncg.edu)

Objective: learn how to format the appearance of your document in your chosen output format.

.....

2.1. Headings

.....

TASK: Read the question below, click **Knit** to render the document and return here to answer the question.

QUESTION: What difference do you notice about the above line that starts with **# Section 2: Formatting a Report Using RMarkdown Syntax** compared to the line **Objective: learn to format how your document appears in the output of your choice?**

ANSWER: The “Section 2...” has a pund sign in front of it while the “objective...” line does not.

Placing a hashtag (#) at the beginning of a line turns it into a heading. RMarkdown supports six levels of headings.

TASK: Click Knit to observe how different heading levels below appear in the rendered output.

```
# Line 1: This is an example to show different levels of headings in RMarkdown.
## Line 2: Line 1: This is an example to show different levels of headings in RMarkdown.
### Line 3: This is an example to show different levels of headings in RMarkdown.
#### Line 4: This is an example to show different levels of headings in RMarkdown.
##### Line 5: This is an example to show different levels of headings in RMarkdown.
##### Line 6: This is an example to show different levels of headings in RMarkdown.
```

QUESTION: What happens to the size of text when you increase the number of hashtags (#)?

ANSWER: As the number of pound signs increased, the text got smaller. Except lines 3-5 seem to have similar size font but different spacing between the lines. Line 6 is not bolded at all.

In the example above, the higher the number of hashtags, the lower the heading level (smaller font).

QUESTION: How many hashtags would you use if you wanted highest level heading (largest font)?

ANSWER: One.

QUESTION: In the rendered output, do Line 5 and Line 6 from above example appear in the same line?

ANSWER:yes. Line 5 is bolded and line 6 is not.

TASK: Go to the metadata for this assignment and change the output from `pdf_document` to `html_document`.

QUESTION: Did your Line 5 and Line 6 appear in different lines?

ANSWER:Yes, now they are on different lines.

Sometimes the way the program renders output differs between formats. You can leave two spaces at the end of the line for a line break.

TASK: Change the output back to `pdf_document`.

TASK: **Go back** to all the above example lines “Line 1” thru “Line 6” within single set of three backticks “```”. Go back above and do as below:

```
# Line 1: This is an example to show different levels of headings in RMarkdown.
## Line 2: Line 1: This is an example to show different levels of headings in RMarkdown.
### Line 3: This is an example to show different levels of headings in RMarkdown.
#### Line 4: This is an example to show different levels of headings in RMarkdown.
##### Line 5: This is an example to show different levels of headings in RMarkdown.
##### Line 6: This is an example to show different levels of headings in RMarkdown.
```

.....

2.2 Italics

.....

You may sometimes want to italicize text. RMarkdown allows you to do this easily. To make your text italics, you can enclose it within single asterisk `*[text]*`.

Line 1: This is an example to show how to make text italics.

TASK: Rewrite Line 1 below and only italicize the word “italics.” Leave two spaces at the end of sentence for a line break.

ANSWER: Line 1: This is an example to show how to make text *italicized*.

You may be wondering if there is another way to make the text italics, and in fact, there is. You can also enclose your text within single underscore `_[text]_`.

TASK: In the paragraph below, make “Arnav,” “Biological Data Wrangling and Visualization” and “RMarkdown” italics in the following paragraph using both the methods (asterisks and underscore) mentioned above at least once.

My name is *Arnav* Pandey. I am currently enrolled in *Biological Data Wrangling and Visualization* (BIO 457/657). This is a TASK: to use *RMarkdown* to make a text output italics.

.....

2.3 Bold

.....

Sometimes you may want to bold your text to emphasize its importance. RMarkdown supports that as well.

TASK: Read the question below, click Knit, and return to answer the question below.

QUESTION: Which of the following lines gives you bold text?

Line 1. This is an example to show how to make text bold.

Line 2: This is an example to show how to make text bold.

ANSWER: Line 2 is bolded.

You can use two asterisks before and after the text to make it bold (****[text]****).

There is also another way to make a text bold by enclosing your text within two underscores(**__[text]__**).

Don't believe me? Knit to see if the line below is bold.

Line 3: This is an example to show how to make text bold.

Like you practiced with italics, you can make any word or phrase within a sentence bold. See example of it below when you hit Knit.

Line 5: This is an example to show how to make text **bold**.

Line 4: This is an **example** to show how to make text bold.

TASK: In the paragraph below, make “Arnav,” “Biological Data Wrangling and Visualization” and “RMarkdown” bold in the following paragraph using both the methods (asterisks and underscores) mentioned above at least once.

My name is **Arnav** Pandey. I am currently enrolled in **Biological Data Wrangling and Visualization** (BIO 457/657). This is a TASK: to use **RMarkdown** to make a text output bold.

.....

2.4 Bold and Italics

.....

Is there a way to make a text both bold and italics at the same time? Indeed, there is! You can enclose your text within three asterisks (*****[text]****). or underscores (___[text]___).

TASK: In the paragraph below, make “Arnav,” “Biological Data Wrangling and Visualization” and “RMarkdown” bold italics using both the methods mentioned above at least once.

My name is *Arnav* Pandey. I am currently enrolled in *Biological Data Wrangling and Visualization* (BIO 457/657). This is a TASK: to use *RMarkdown* to make a text output bold.

In sections 2.2, 2.3, and 2.4, you learned how to make text italics, bold, or bold+italics by enclosing the text within one, two, and three asterisks or underscores, respectively. You might be tempted to start enclosing a text by underscore and end it using asterisk. For example:

```
*italics_  
___bold**  
***bold italics___
```

Above lines will not give your desired output, so you will have to be consistent with how you enclose your texts. Use the same symbols at both ends!

.....

2.5 Bulleted list

.....

In RMarkdown, you can list your items in the RMarkdown as an unordered (bulleted) list.

To do that, you can follow these steps. Step 1: leave an empty line before list. Step 2: Use ‘+’, ‘-’, or ‘*’ followed by single space before you write your list item.

Step 3: To add another item, leave two spaces at the end of the previous item and go to a new line below and repeat Step 2.

TASK: Knit to see how the items below appear in a list in the output.

- Apple
- Banana
- Raspberry

You can also nest subitems within an item. If you want a nested list as such, just indent the subitem that you want within an item with two spaces. Below is an example of it.

TASK: Hit Knit to look at the output of the lines below:

- Apple
- Banana
- Berries
 - Blueberries
 - Strawberries

- Raspberries
 - Blackberries
 - Cranberries
- Papaya

TASK: Copy the above list below, add an item “Melon” and nest different types of melons as its subitems. Hit Knit to see your output.

- Apple
- Banana
- Berries
 - Blackberries
 - Blueberries
 - Cranberries
 - Raspberries
 - Strawberries
- Melons
 - Honeydew
 - Watermelon
- Papaya

.....

2.6 Numbered List

.....

Getting a numbered list is even easier compared to unordered or bulleted list. To generate a numbered list, you can follow these steps. Step 1: Leave an empty line before list. Step 2: Use uppercase/lowercase English alphabets, Arabic numerals, uppercase or lowercase roman numerals followed by space before you write your list item. Be consistent how you want to list. Step 3: To add another item, leave two spaces at the end of the previous item and go to a new line below and repeat Step 2.

- I. Apple
- II. Banana
- III. Raspberry

You can also nest subitems within an item. If you want a nested list as such, just indent the subitem that you want within an item with three spaces. You can use different order format for your subitems than your primary item as well. Below is an example of it. **Please be careful with spaces!**

1. Apple
2. Banana
3. Berries
 1. Blueberries
 2. Strawberries
 3. Raspberries
 4. Blackberries
 5. Cranberries
4. Papaya

Note: If you want to use Roman Numerals, you may have to leave an empty line between items on list.

Section 3: Inserting Code Chunks and Running Basic R Code

Author: Michelle Bautista Canuto (m_bautistac@uncg.edu) We can insert a code chunk by clicking on the green-button with a “c” on the inside and a “+” on the upper left. Click “R”. We can insert functions within the chunk of code. We will see this next.

QUESTION: What is a code chunk? (hint look back at the slides in Canvas) ANSWER The chunk of code that is written in between the “`````” marks.

TASK: Practice inserting a code chunk. Run a simple calculation such as “2+2” below {r}. Run this code by clicking on the green side-triangle button:

```
2+2
```

```
## [1] 4
```

Testing in line hyperlinks for the last sections [R Markdown documentation] (<https://rmarkdown.rstudio.com>).

Next, let’s assign variables. Run the following variables:

```
x<-10  
y<-5
```

To view the contents of the variable x, just type ‘x’ and run

```
x
```

```
## [1] 10
```

TASK: Write a chunk of code multiplying the variables x and y.


```
x*y
```

```
## [1] 50
```

QUESTION: What was the result? ANSWER: 50

We will analyze the built-in 'mtcars' dataset. Run the following chunk of code.

```
head(mtcars)
```

```
##           mpg  cyl  disp  hp  drat    wt   qsec vs  am  gear  carb
## Mazda RX4      21.0   6  160 110  3.90  2.620 16.46  0   1    4    4
## Mazda RX4 Wag  21.0   6  160 110  3.90  2.875 17.02  0   1    4    4
## Datsun 710      22.8   4  108  93  3.85  2.320 18.61  1   1    4    1
## Hornet 4 Drive  21.4   6  258 110  3.08  3.215 19.44  1   0    3    1
## Hornet Sportabout 18.7   8  360 175  3.15  3.440 17.02  0   0    3    2
## Valiant        18.1   6  225 105  2.76  3.460 20.22  1   0    3    1
```

QUESTION: What do you think the head() function does? ANSWER: It prints the first portion of the dataset so you can view the first few columns and rows.

To display the summary statistics for the dataset use the summary() function. Run the following chunk of code.

```
summary(mtcars)
```

```
##           mpg           cyl           disp           hp
##  Min.   :10.40  Min.   :4.000  Min.   : 71.1  Min.   : 52.0
## 1st Qu.:15.43  1st Qu.:4.000  1st Qu.:120.8  1st Qu.: 96.5
## Median :19.20  Median :6.000  Median :196.3  Median :123.0
## Mean   :20.09  Mean   :6.188  Mean   :230.7  Mean   :146.7
## 3rd Qu.:22.80  3rd Qu.:8.000  3rd Qu.:326.0  3rd Qu.:180.0
## Max.   :33.90  Max.   :8.000  Max.   :472.0  Max.   :335.0
##           drat           wt           qsec           vs
##  Min.   :2.760  Min.   :1.513  Min.   :14.50  Min.   :0.0000
## 1st Qu.:3.080  1st Qu.:2.581  1st Qu.:16.89  1st Qu.:0.0000
## Median :3.695  Median :3.325  Median :17.71  Median :0.0000
## Mean   :3.597  Mean   :3.217  Mean   :17.85  Mean   :0.4375
## 3rd Qu.:3.920  3rd Qu.:3.610  3rd Qu.:18.90  3rd Qu.:1.0000
## Max.   :4.930  Max.   :5.424  Max.   :22.90  Max.   :1.0000
##           am           gear           carb
##  Min.   :0.0000  Min.   :3.000  Min.   :1.000
## 1st Qu.:0.0000  1st Qu.:3.000  1st Qu.:2.000
## Median :0.0000  Median :4.000  Median :2.000
## Mean   :0.4062  Mean   :3.688  Mean   :2.812
## 3rd Qu.:1.0000  3rd Qu.:4.000  3rd Qu.:4.000
## Max.   :1.0000  Max.   :5.000  Max.   :8.000
```

Within a new chunk of code, we will code to import the entire mtcars dataset. Run the following code by clicking on the green side-triangle button for "Run Current Chunk"

```
data(mtcars)
```

QUESTION: After running the previous code, view the dataframe of mtcars. How many variables are in the mtcars dataset? ANSWER: There are 11 variables.

Or you can view the dataset in the console by typing “mtcars” and click ‘Run Selected Lines’. mtcars

To find the number of rows in the dataset, we can use the nrow() function within a new chunk of code. Run the following code.

```
nrow(mtcars)
```

```
## [1] 32
```

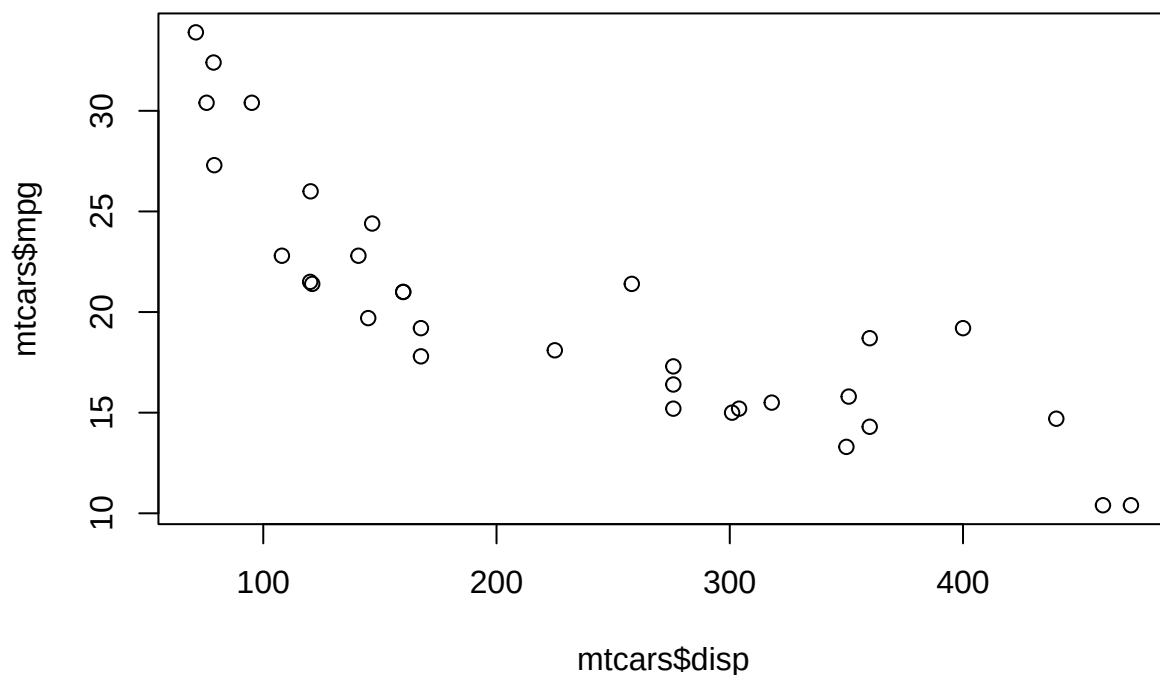
TASK: Write a code to find the number of columns in the ‘mtcars’ dataset using the ncol() function. Remember to use a new chunk of code.

```
ncol(mtcars)
```

```
## [1] 11
```

QUESTION: What is the result showing in your script? ANSWER: 11

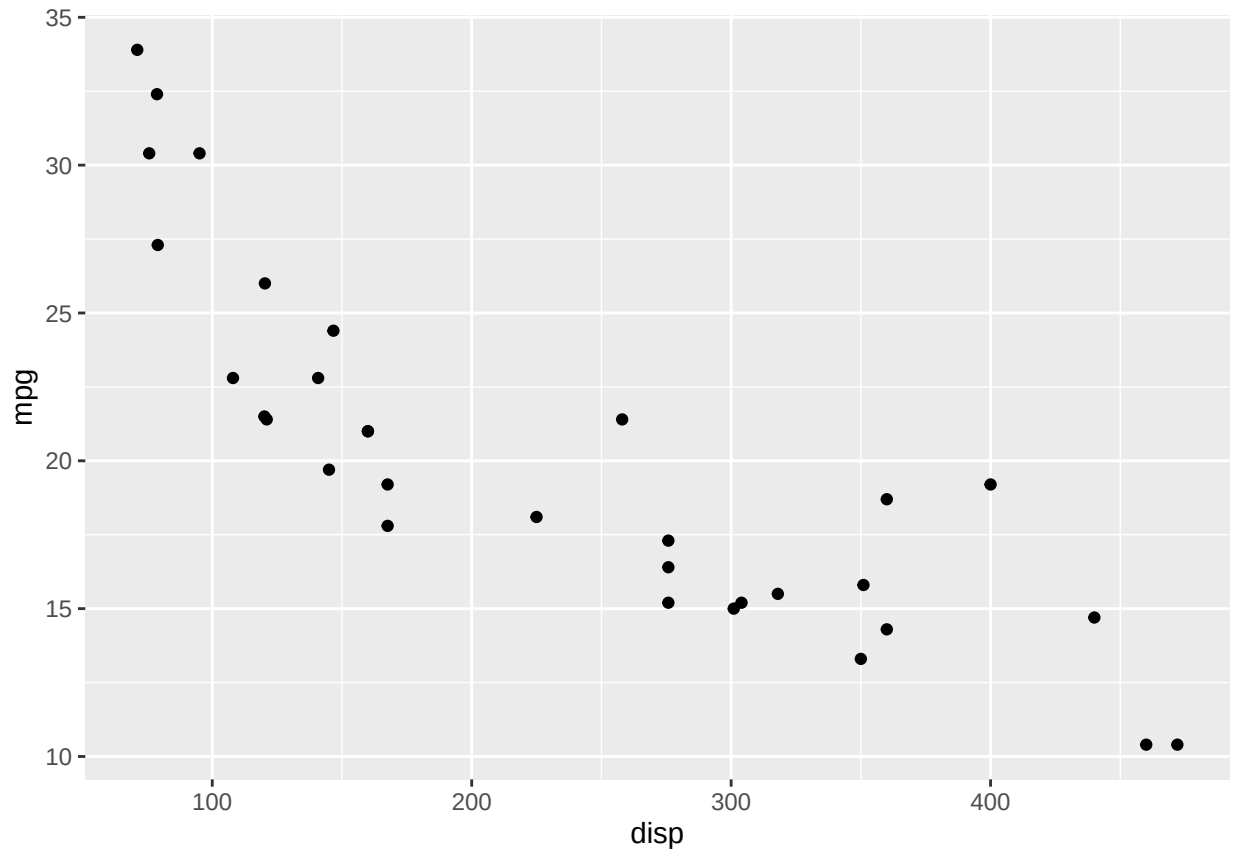
The ‘disp’ column in the dataset stands for engine displacement. Engine displacement tells you how big the engine is in cubic inches. The ‘mpg’ column stands for fuel efficiency in miles per gallon. Lets plot a graph showing fuel efficiency over displacement for each car. Run the following code.



QUESTION: What trend is the plot showing? ANSWER: The bigger the engine, the lower mpg.

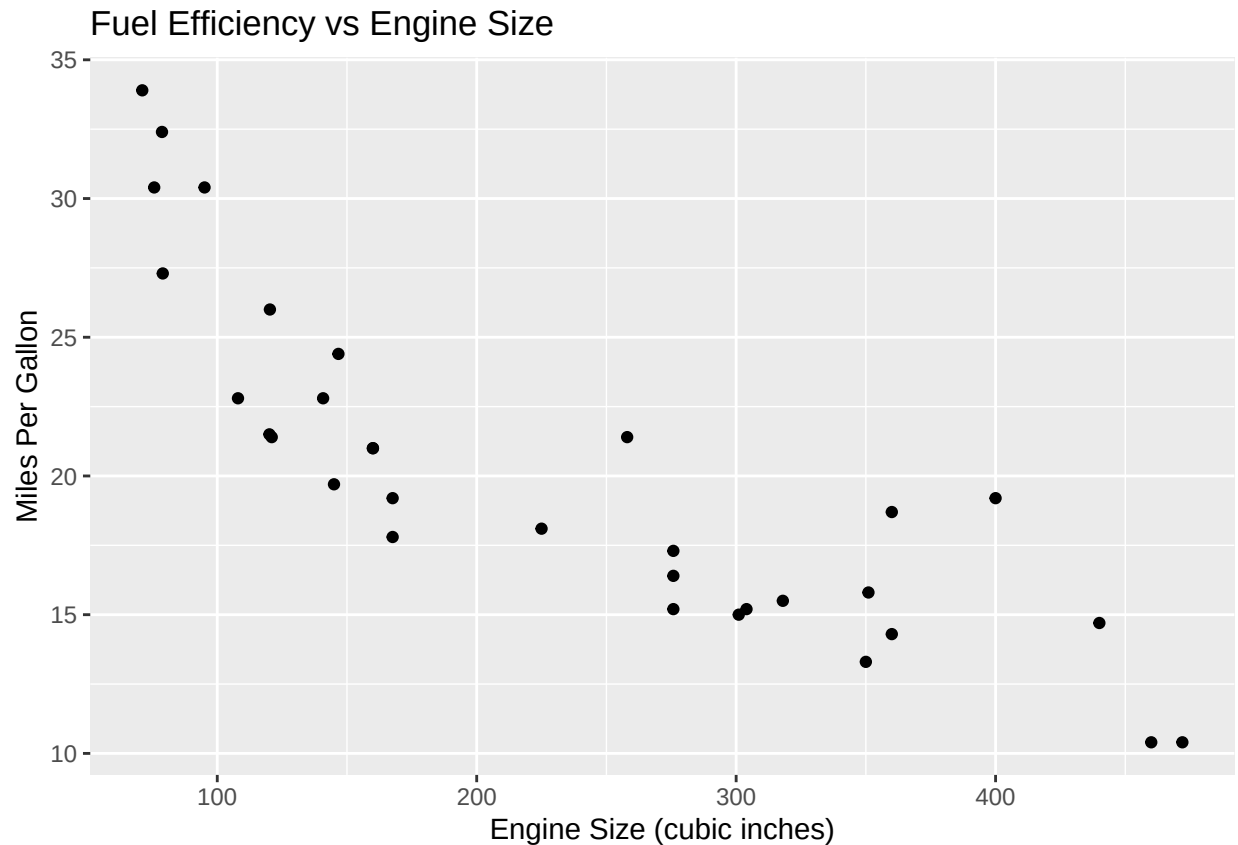
We can also use ggplot to run this within our chunk of code. This permits our plot to be displayed in the final report once “knit”.

```
library(ggplot2)
ggplot(mtcars, aes(x=disp, y=mpg)) + geom_point()
```



Let's change the labels of the x-axis to 'Engine Size (cubic inches)' Let's change the y-axis to 'Miles Per Gallon' Let's add a title called "Fuel Efficiency vs Engine Size".

```
ggplot(mtcars, aes(x=disp, y=mpg)) + geom_point() +
  labs(
    x="Engine Size (cubic inches)",
    y="Miles Per Gallon",
    title="Fuel Efficiency vs Engine Size"
  )
```



We could add a legend to show which car each point corresponds to. But first, we should add a column displaying the car names using dplyr.

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

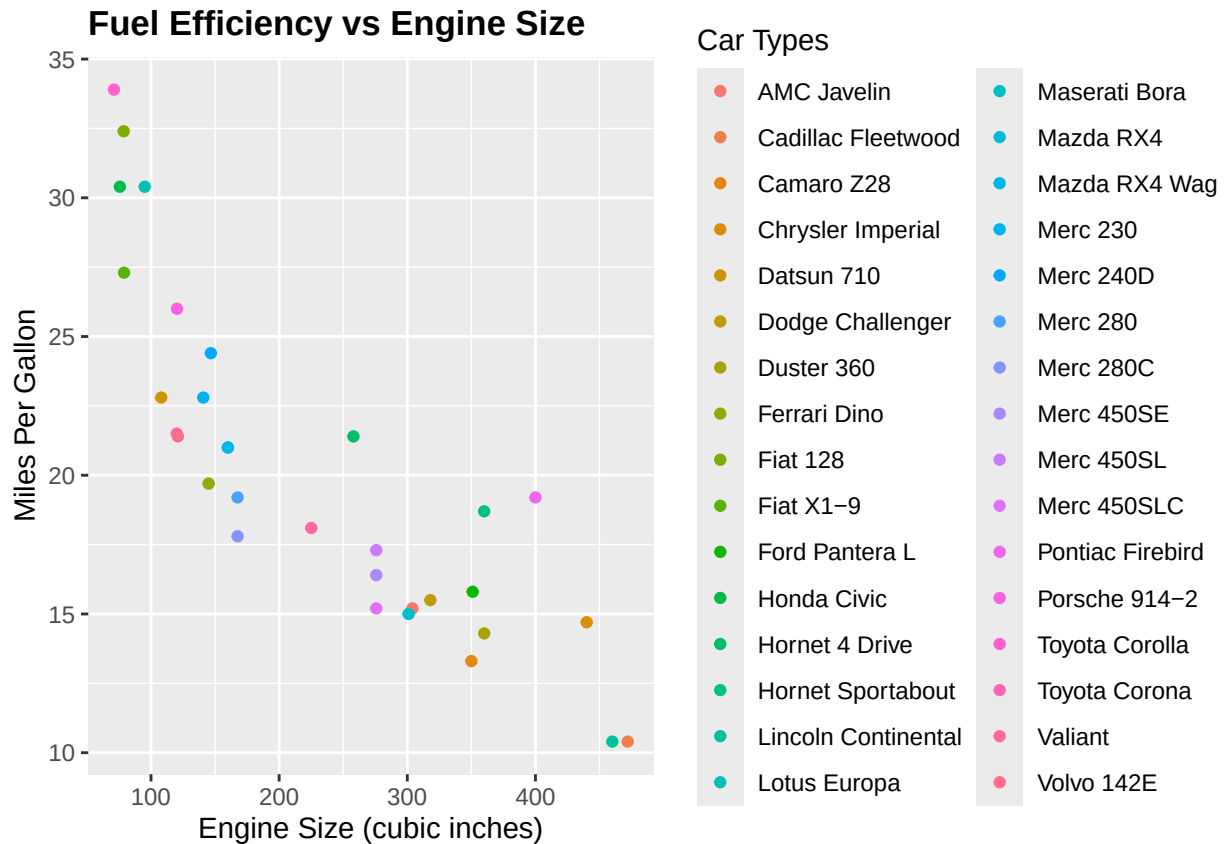
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
mtcars<-mtcars %>%
  mutate(car_name=rownames(mtcars))
```

Lets'bold' the title Let's add a legend to differentiate each point.

```
ggplot(mtcars,aes(x=disp,y=mpg, color=factor(car_name)))+
  geom_point()+
  labs(
```

```
x="Engine Size (cubic inches)",
y="Miles Per Gallon",
title="Fuel Efficiency vs Engine Size",
color="Car Types")+
theme(plot.title = element_text(face="bold"))
```



Lets organize the fuel efficiency from highest to lowest. Run the following.

```
mtcars %>%
  select(mpg, disp) %>%
  arrange(desc(mpg))
```

```
##           mpg  disp
## Toyota Corolla  33.9  71.1
## Fiat 128        32.4  78.7
## Honda Civic     30.4  75.7
## Lotus Europa    30.4  95.1
## Fiat X1-9       27.3  79.0
## Porsche 914-2   26.0 120.3
## Merc 240D       24.4 146.7
## Datsun 710      22.8 108.0
## Merc 230        22.8 140.8
## Toyota Corona   21.5 120.1
## Hornet 4 Drive  21.4 258.0
## Volvo 142E      21.4 121.0
```

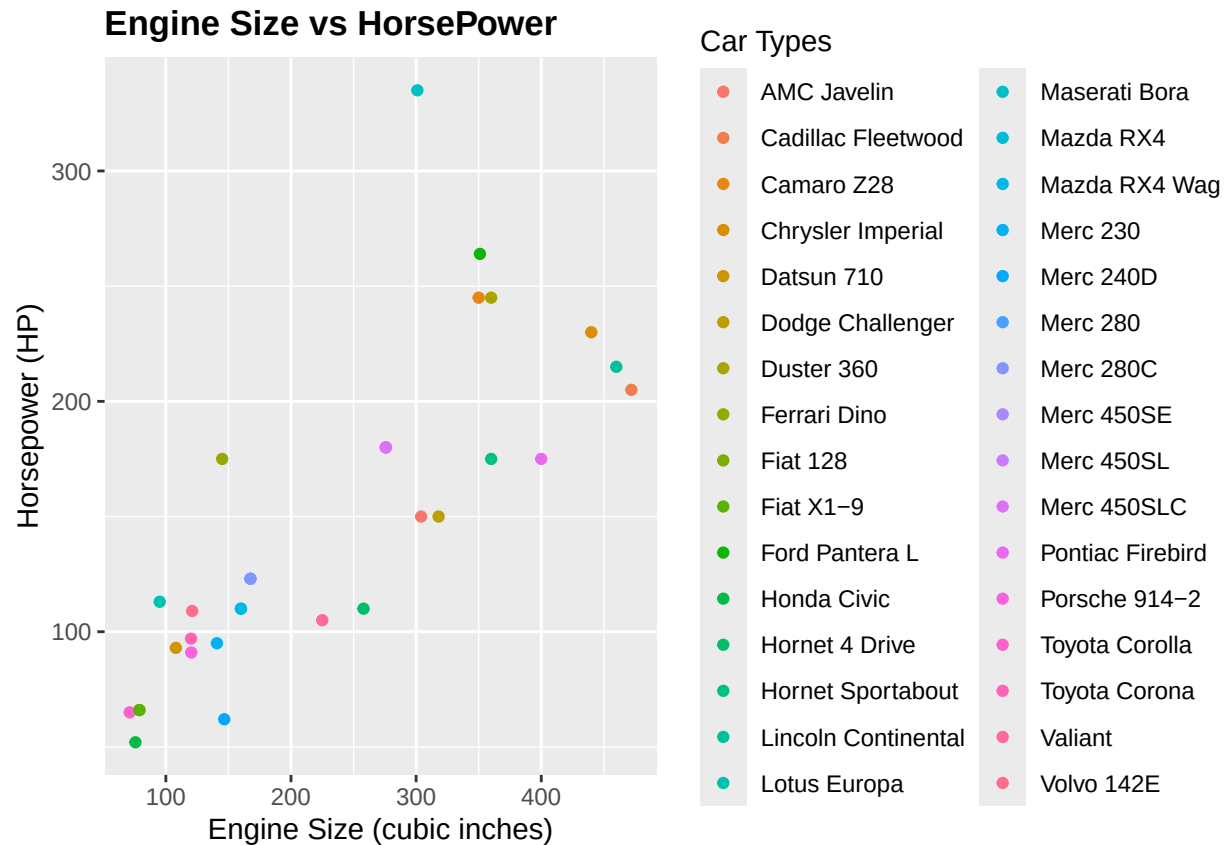
## Mazda RX4	21.0	160.0
## Mazda RX4 Wag	21.0	160.0
## Ferrari Dino	19.7	145.0
## Merc 280	19.2	167.6
## Pontiac Firebird	19.2	400.0
## Hornet Sportabout	18.7	360.0
## Valiant	18.1	225.0
## Merc 280C	17.8	167.6
## Merc 450SL	17.3	275.8
## Merc 450SE	16.4	275.8
## Ford Pantera L	15.8	351.0
## Dodge Challenger	15.5	318.0
## Merc 450SLC	15.2	275.8
## AMC Javelin	15.2	304.0
## Maserati Bora	15.0	301.0
## Chrysler Imperial	14.7	440.0
## Duster 360	14.3	360.0
## Camaro Z28	13.3	350.0
## Cadillac Fleetwood	10.4	472.0
## Lincoln Continental	10.4	460.0

QUESTION: Which car has the highest fuel efficiency? ANSWER: Toyota Corolla

QUESTION: What is the size of it's engine? ANSWER: 71.1

TASK: Use code to create a plot comparing disp vs hp from the 'mtcars' dataset. With the following:
 Use the 'disp' data for the x-axis and labeled 'Engine Size (cubic inches)' Use the 'hp' data for the y-axis and labeled 'Horsepower (HP)' Add an appropriate title for the plot with a 'bold' font Add a legend colored by 'car_name' and labeled "Car Types"

```
ggplot(mtcars, aes(x = disp, y = hp, color = car_name)) +
  geom_point() +
  labs(x = "Engine Size (cubic inches)",
       y = "Horsepower (HP)",
       title = "Engine Size vs HorsePower",
       color = "Car Types") +
  theme(plot.title = element_text(face="bold"))
```



Lets organize the horsepower from highest to lowest. Run the following.

```
mtcars %>%
  select(hp, disp) %>%
  arrange(desc(hp))
```

```
##           hp  disp
## Maserati Bora    335 301.0
## Ford Pantera L   264 351.0
## Duster 360       245 360.0
## Camaro Z28       245 350.0
## Chrysler Imperial 230 440.0
## Lincoln Continental 215 460.0
## Cadillac Fleetwood 205 472.0
## Merc 450SE       180 275.8
## Merc 450SL       180 275.8
## Merc 450SLC      180 275.8
## Hornet Sportabout 175 360.0
## Pontiac Firebird 175 400.0
## Ferrari Dino     175 145.0
## Dodge Challenger 150 318.0
## AMC Javelin      150 304.0
## Merc 280         123 167.6
## Merc 280C        123 167.6
## Lotus Europa     113  95.1
## Mazda RX4        110 160.0
```

## Mazda RX4 Wag	110	160.0
## Hornet 4 Drive	110	258.0
## Volvo 142E	109	121.0
## Valiant	105	225.0
## Toyota Corona	97	120.1
## Merc 230	95	140.8
## Datsun 710	93	108.0
## Porsche 914-2	91	120.3
## Fiat 128	66	78.7
## Fiat X1-9	66	79.0
## Toyota Corolla	65	71.1
## Merc 240D	62	146.7
## Honda Civic	52	75.7

QUESTION: What trend is the plot showing between engine size and horsepower? ANSWER: As engine size increases, so does horsepower.

QUESTION: Which cars have the highest and lowest hp? ANSWER: Maserati Bora

Don't forget to save and commit your changes.

Section 4: Add Visualizations and Tables to Your Report

Author: Themesha Parker (tlparker@uncg.edu)

Goal

In this section, you will create graphs and a summary table using the mtcars data set. You will also practice describing patterns that you observe in the output.

Before beginning, make sure you can access the mtcars dataset. Run the line below to preview the first few rows.

```
head(mtcars)
```

Also make sure you have Tidyverse installed! Run the line of code below.

```
library(tidyverse)
```

Part A: Create a Scatterplot

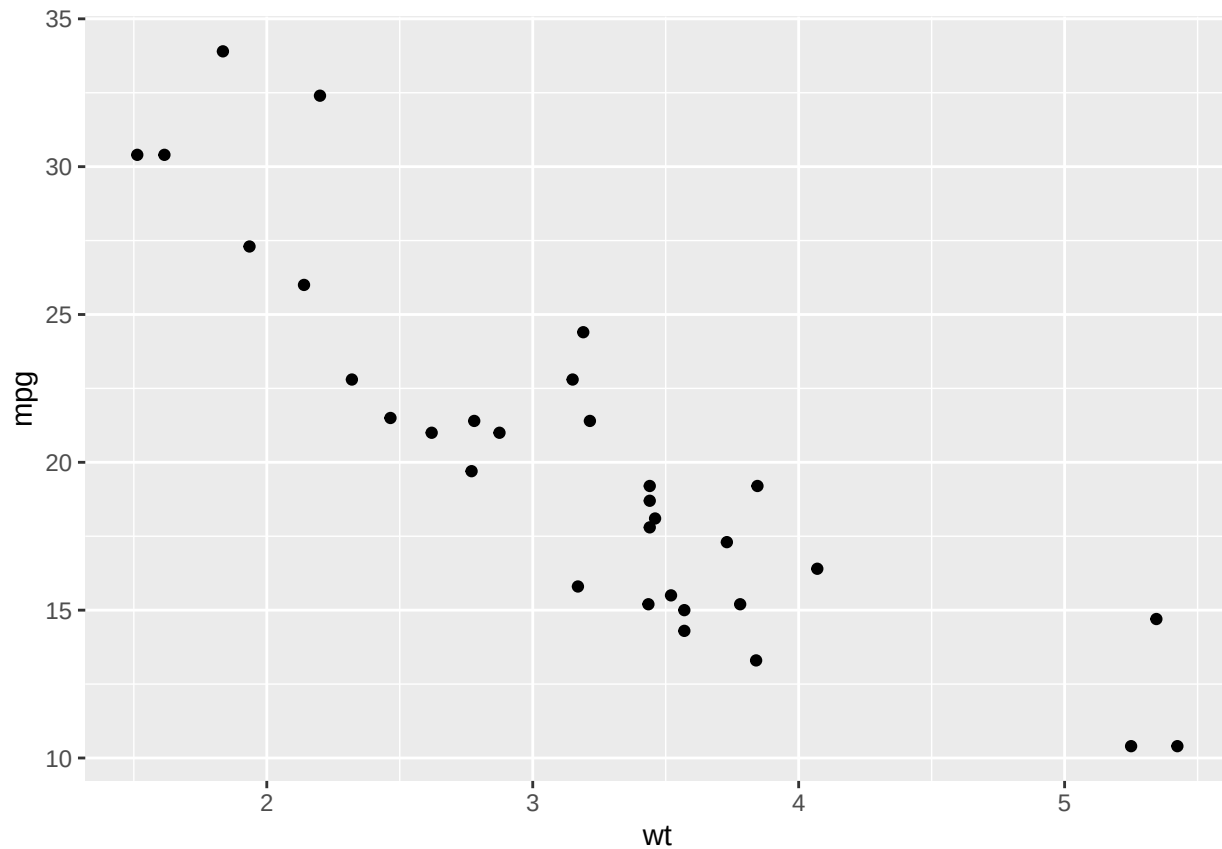
The variables wt and mpg in the mtcars data set can help us explore the relationship between car weight and fuel efficiency.

TASK:

Create a scatter plot with wt on the x-axis and mpg on the y-axis.

Hint: use the plot() function

```
# Write your code below this line
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point()
```

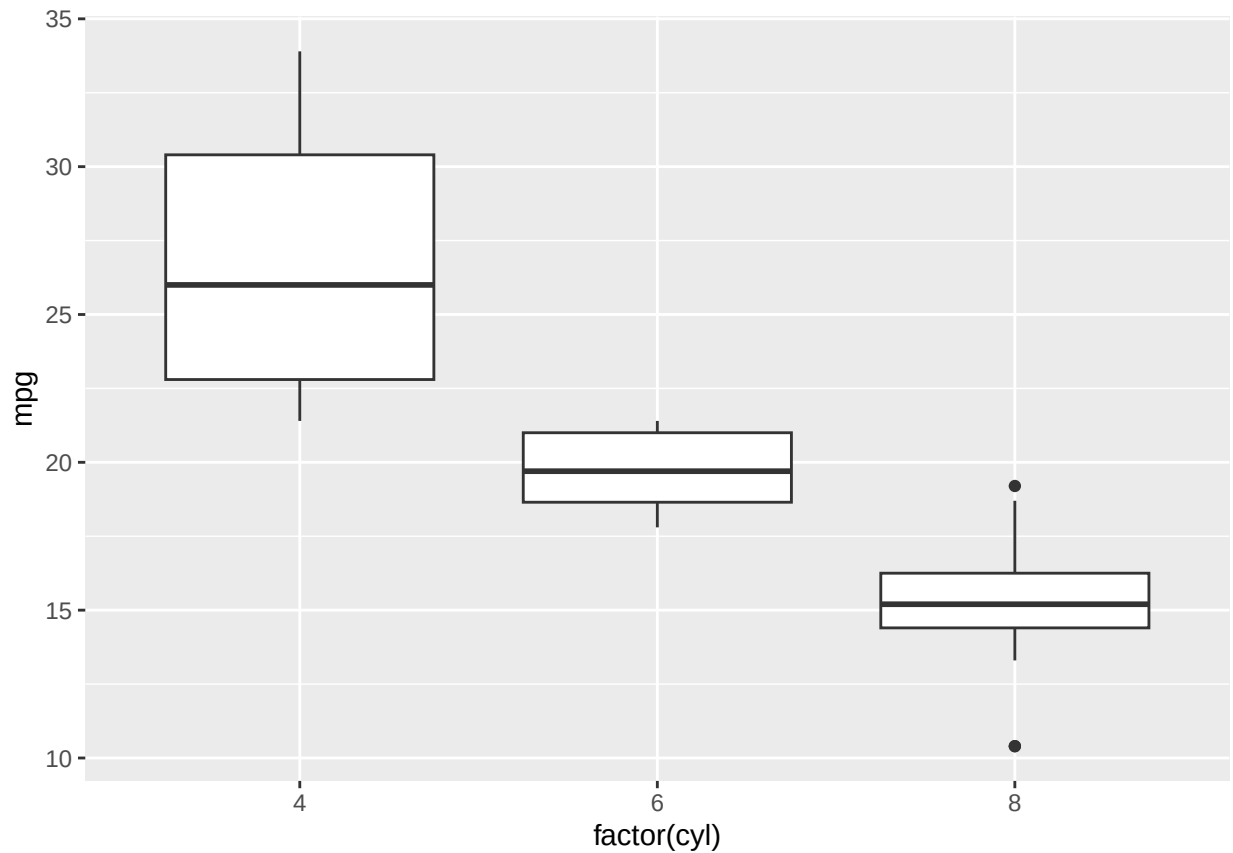



TASK:

Modify your scatterplot so that it includes: a title, an x-axis label, a y-axis label, a point color of your choice

Hint: Think back to previous assignments(`main=`, `xlab=`, `ylab=`, and `col=`)

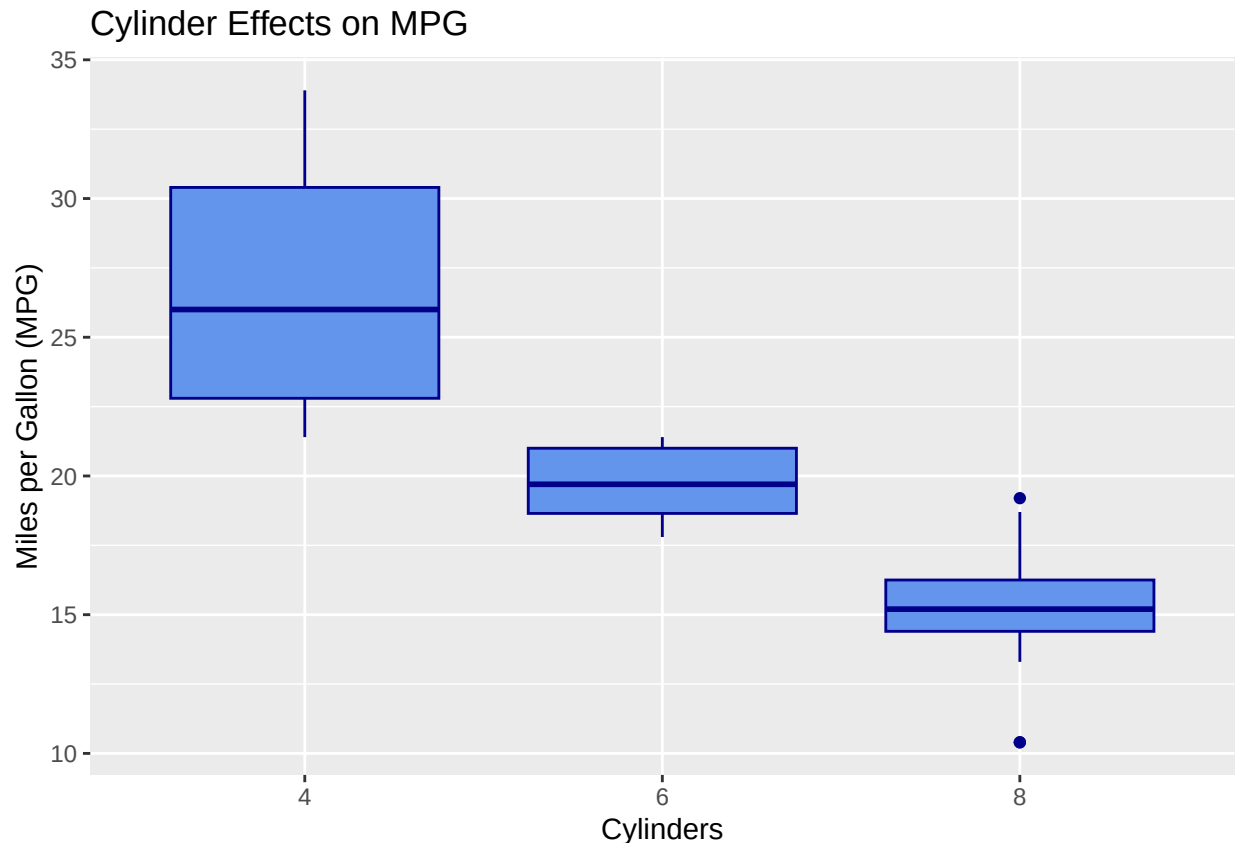
```
# Write your updated scatterplot code below this line
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point(color = "lightpink3") +
  labs(x = "Car Weight (in 1000lbs)",
       y = "Miles per Gallon (MPG)",
       title = "Car Weight vs MPG")
```

TASK:

Modify your boxplot so that it includes: a title, an x-axis label, a y-axis label, and a fill color of your choice

```
# Write your updated boxplot code below this line
ggplot(mtcars, aes(x = factor(cyl), y = mpg)) +
  geom_boxplot(color = "darkblue", fill = "cornflowerblue") +
  labs(x = "Cylinders",
       y = "Miles per Gallon (MPG)",
       title = "Cylinder Effects on MPG")
```



QUESTION: What does each box in this plot represent? ANSWER: Cars with different numbers of cylinders.

QUESTION: Which cylinder group appears to have the highest fuel efficiency? ANSWER: 4 cylinder.

QUESTION: Which cylinder group appears to have the lowest fuel efficiency? ANSWER: 8 cylinder.

QUESTION: Based on the boxplot, how does fuel efficiency seem to change as the number of cylinders increases? ANSWER: As number of cylinders increases, fuel efficiency decreases.

Part C: Create a Summary Table

You will now create a table showing the average miles per gallon for each cylinder group.

TASK:

Use dplyr to calculate the average mpg for each value of cyl. Rename the data avg_mpg_by_cyl

Hint: you need to group the data by cyl then summarize the mean of mpg.

Use the space below to write your code:

```
# Write your code below this line
avg_mpg_by_cyl <- mtcars %>%
  group_by(cyl) %>%
  summarise(avg_mpg = mean(mpg)) %>%
  ungroup()
```

TASK:

Display your results as a clean table.

Use the space below to write your code:

```
# Write your code below this line
(avg_mpg_by_cyl)
```

```
## # A tibble: 3 x 2
##   cyl avg_mpg
##   <dbl>   <dbl>
## 1     4    26.7
## 2     6    19.7
## 3     8    15.1
```

QUESTION: What does this table show? ANSWER: Average miles per gallon for the cylinder groups.

QUESTION: Which cylinder group has the highest average mpg? ANSWER:4.

QUESTION: Which cylinder group has the lowest average mpg? ANSWER:8.

Part D: Compare the Visuals

Now that you have created both graphs and the table, compare the information they provide.

QUESTION: How does the scatterplot help you understand the relationship between weight and fuel efficiency? ANSWER: Scatterplots show the data in continuous variables. It shows us as the weight of the car increases, the fuel efficiency decreases.

QUESTION: How does the boxplot help you compare different groups of cars? ANSWER: The boxplot shows us the differences across quartiles and shows us the medians of each group.

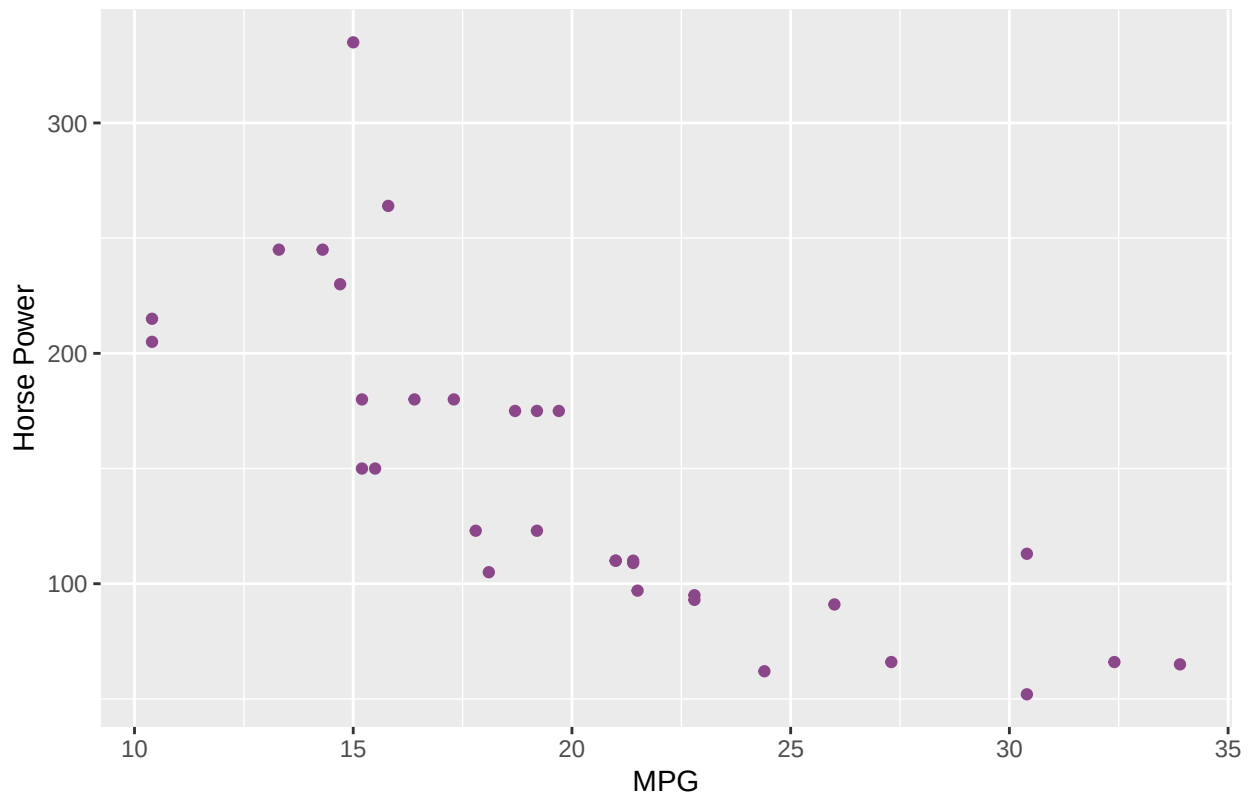
Part E: Now you try

1. Use a different variable from mtcars to make an additional scatter plot.
2. Change the color of your scatterplot.

Write your code below:

```
# Write your code below this line
ggplot(mtcars, aes(x = mpg, y = hp)) +
  geom_point(color = "orchid4") +
  labs(x = "MPG",
       y = "Horse Power",
       title = "How much horse power was at the Kentucky Derby?")
```

How much horse power was at the Kentucky Derby?



QUESTION: What did you change, and how did it affect the output? ANSWER:I changed the x-axis to MPG and the y-axis to horse power. As horse power decreases, MPG increases.

Section 5: Improve and Finalize Your R Markdown Report

Objective: Learn how to make your report more polished, readable, and professional. In this section, we will take everything we have learned and improve the overall quality of our R Markdown report so it is easier to read and more useful.

Author: Dhanush Devanand (s_devanand@uncg.edu)

Step 1: Understanding Inline R Code

So far, we have been writing code in chunks. However, sometimes we want to include results directly inside our text. **Inline R code** allows us to insert calculations into sentences without a separate code chunk.

TASK:

Copy and paste the following lines directly into your R Markdown document, **outside** of any code chunk, and then Knit:

```
The mtcars dataset has 32 rows and 12 columns.  
The average miles per gallon (mpg) is 20.09.
```

Check

```
2+2
```

```
## [1] 4
```

The mtcars dataset has 32 rows and 12 columns. The average miles per gallon (mpg) is 20.09.

Did the numbers appear automatically in your rendered sentence? yes

QUESTION: What is inline R code, and how is it different from a regular code chunk? ANSWER:Inline code gives you the answer to the code in the sentence of your finalized markdown report.

QUESTION: Why is inline R code useful when working with a dataset that might be updated or changed over time? ANSWER:You don't have to go change all the sentences everytime you can just refresh the file.

TASK:

Now try writing your own inline R code sentence. Using the mtcars dataset, write a sentence that includes the maximum horsepower value using inline R code. The function `max()` will be helpful here.

Your sentence: The Maserati Bora has the highest horsepower of `max(mtcars$hp)`.

Why This Matters

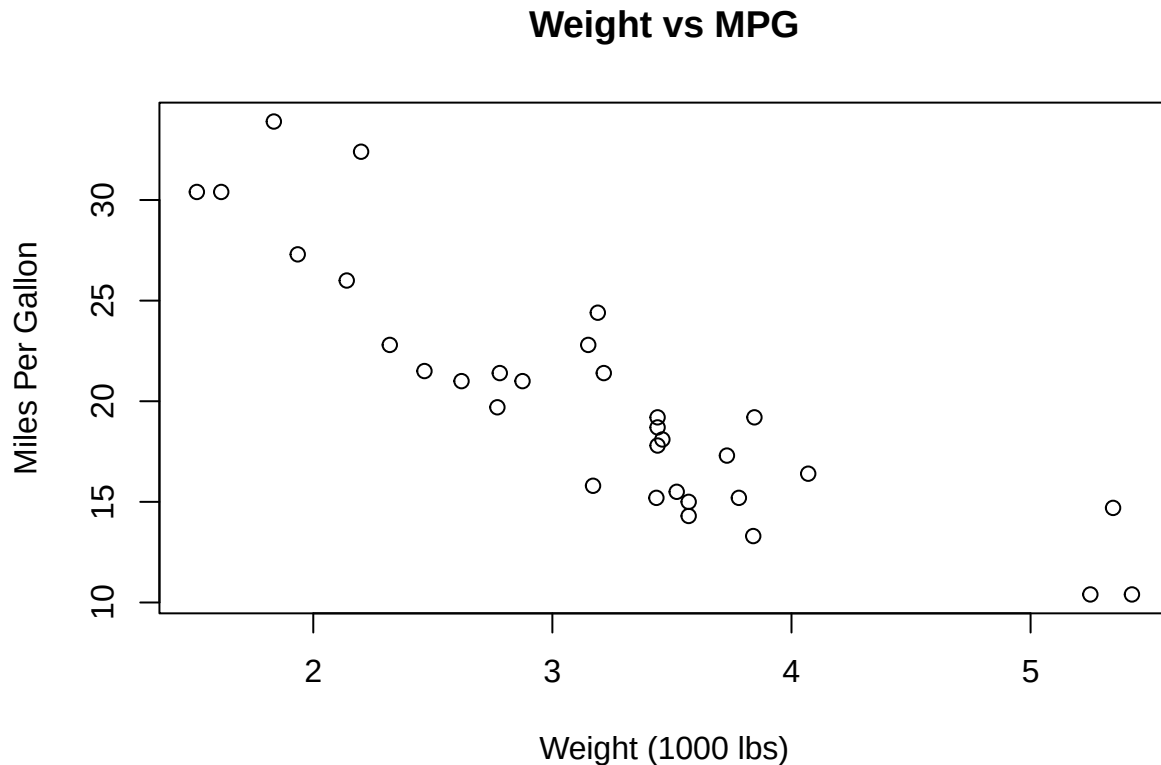
Inline R code makes your report dynamic. If the dataset changes, your results will automatically update without you needing to rewrite anything. This is especially important in professional or scientific reports where accuracy matters.

Step 2: Improving Readability by Hiding Code

Sometimes, showing all code can make a report look cluttered. If we only care about the output — like a graph — we can hide the code using the `echo=FALSE` chunk option.

TASK:

The chunk below currently shows the code when knitted. Add `echo=FALSE` inside the curly braces so that only the plot appears in the output, not the code. Then Knit your document.



Hint: Your chunk header should look like this when you are done: ````{r echo=FALSE}`

Check

Do you see the plot in the output but **not** the code that created it? Yup!

QUESTION: Why might you want to hide code in a final report? ANSWER: To make the document look cleaner and so the reader does not get overwhelmed or distracted.

QUESTION: Can you think of a situation where you would want to **show** your code in a report rather than hide it? Who might be reading that report? ANSWER: The reviewers of a paper might want to see the code or others that you are working on the project with.

TASK:

Now apply `echo=FALSE` to the ggplot chunk you created back in Section 3. Knit the document and confirm only the plot appears. There are lots of plots in section 3 so I just did the first plot. This is also mainly because when I used `echo=false` for all of them, the plots were cut off the page??

Step 3: Removing Warnings and Messages

When you load packages or run certain functions, R sometimes prints extra warnings and messages that are not important to the reader. These can make your report look messy. You can suppress them using the chunk options `message=FALSE` and `warning=FALSE`.

TASK: — Part A: See the messages first

Run the chunk below **without** any chunk options and Knit. Read through the output carefully and write down any extra messages or warnings you notice below.

```
library(tidyverse)
summary(mtcars)
```

```
##      mpg      cyl      disp      hp
## Min.   :10.40   Min.    :4.000   Min.    : 71.1   Min.    : 52.0
## 1st Qu.:15.43   1st Qu.:4.000   1st Qu.:120.8   1st Qu.: 96.5
## Median :19.20   Median :6.000   Median :196.3   Median :123.0
## Mean   :20.09   Mean    :6.188   Mean    :230.7   Mean    :146.7
## 3rd Qu.:22.80   3rd Qu.:8.000   3rd Qu.:326.0   3rd Qu.:180.0
## Max.   :33.90   Max.    :8.000   Max.    :472.0   Max.    :335.0
##      drat      wt      qsec      vs
## Min.   :2.760   Min.    :1.513   Min.    :14.50   Min.    :0.0000
## 1st Qu.:3.080   1st Qu.:2.581   1st Qu.:16.89   1st Qu.:0.0000
## Median :3.695   Median :3.325   Median :17.71   Median :0.0000
## Mean   :3.597   Mean    :3.217   Mean    :17.85   Mean    :0.4375
## 3rd Qu.:3.920   3rd Qu.:3.610   3rd Qu.:18.90   3rd Qu.:1.0000
## Max.   :4.930   Max.    :5.424   Max.    :22.90   Max.    :1.0000
##      am      gear      carb      car_name
## Min.   :0.0000   Min.    :3.000   Min.    :1.000   Length:32
## 1st Qu.:0.0000   1st Qu.:3.000   1st Qu.:2.000   Class :character
## Median :0.0000   Median :4.000   Median :2.000   Mode  :character
## Mean   :0.4062   Mean    :3.688   Mean    :2.812
## 3rd Qu.:1.0000   3rd Qu.:4.000   3rd Qu.:4.000
## Max.   :1.0000   Max.    :5.000   Max.    :8.000
```

What messages or warnings did you see? message about loading tidyverse.

TASK: — Part B: Suppress the messages

Now add `message=FALSE` and `warning=FALSE` to the chunk header above and Knit again.

Hint: Your chunk header should look like this when you are done: ````{r message=FALSE, warning=FALSE}`

Check

Did the extra messages disappear from your rendered output? yes.

QUESTION: What is the difference between `message=FALSE` and `warning=FALSE`? When might you need to use both at the same time? ANSWER: A message is not the same as a warning. You might need both because you could have a message and a warning and by only suppressing one, you may still have unwanted text in the report.

QUESTION: Is it always a good idea to suppress warnings? Can you think of a case where seeing a warning might actually be important? ANSWER: If you don't know why your code is wrong you can keep the warning and send it to collaborators and maybe they can help.

Why This Matters

Hiding unnecessary messages keeps your report clean and professional, especially when sharing results with someone who does not need to see behind-the-scenes R output. However, always make sure you understand what a warning means before choosing to hide it.

Step 4: Hiding Code AND Output with `include=FALSE`

In Step 2, you learned that `echo=FALSE` hides the code but still shows the output. Sometimes you may want to run a chunk silently — hiding both the code and the output entirely. This is done with `include=FALSE`.

This is useful for setup steps like loading libraries or defining variables that need to run but do not need to appear in the report.

TASK:

Run the chunk below as-is and Knit. Note what appears in the output. Then change the chunk option from `echo=FALSE` to `include=FALSE` and Knit again.

Hint: Your chunk header should look like this when you are done: ````${r include=FALSE}`

The average miles per gallon across all cars is 20.09 and the most powerful engine produces 335 horsepower

Check

When using `include=FALSE`, does anything from this chunk appear in the rendered output? Nope!

QUESTION: What is the difference between `echo=FALSE` and `include=FALSE`? ANSWER: `Echo=false` hides the code, but shows the product. `Include=false` hides everything.

QUESTION: Give an example of when you would use `include=FALSE` in a real report. ANSWER: When you are tidying the data to make more usable dataframes.

TASK:

Now use the `my_summary` object created in the chunk above inside an inline R code sentence below. Even though the chunk is hidden, the object is still available to use. Copy and paste the lines below **outside** of a code chunk and Knit to confirm the values appear.

The average miles per gallon across all cars is 20.09 and the most powerful engine produces 335 horsepower

Check

Did the inline values populate correctly even though the chunk that created them was hidden? Yes.

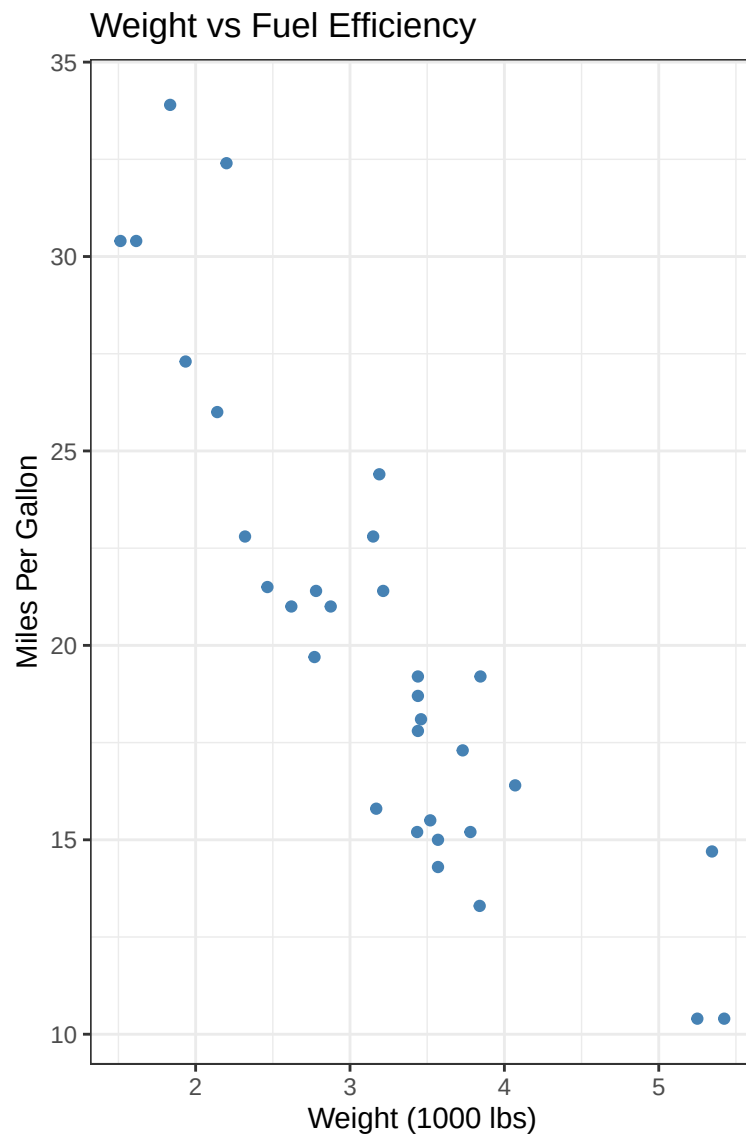
Step 5: Controlling Figure Size

When creating plots in R Markdown, you can control the size of your figures directly in the chunk header using `fig.width` and `fig.height`. This helps make your report look clean and consistent.

TASK:

Run the chunk below and Knit to see the default figure size.

```
ggplot(mtcars, aes(x = wt, y = mpg)) +  
  geom_point(color = "steelblue") +  
  labs(title = "Weight vs Fuel Efficiency",  
        x = "Weight (1000 lbs)",  
        y = "Miles Per Gallon") +  
  theme_bw()
```



TASK:

Now add `fig.width=8` and `fig.height=4` to the chunk header and Knit again.

Hint: Your chunk header should look like this when you are done: ````{r fig.width=8, fig.height=4}`

Check

Did the shape of the plot change in your rendered output? Yes.

QUESTION: How did changing `fig.width` and `fig.height` affect the appearance of your plot? ANSWER: The plot is now wider than it is tall.

QUESTION: Why might consistent figure sizing matter in a professional or academic report? ANSWER: It makes the report look cleaner and cohesive.

TASK:

Try setting `fig.width=4` and `fig.height=6` on the same chunk. Knit and observe how the plot changes. Describe what you notice below.

Observation: The plot is now taller than it is wide.

Step 6: Adding a Table of Contents

A table of contents makes it easier to navigate your report, especially as it gets longer. You can add one by modifying the YAML header at the very top of your `.Rmd` file.

TASK:

Make sure the YAML header at the very top of your document looks like this, then Knit:

```
---
title: "Group5_RMarkdown"
author: "Arnav Pandey, Themesha Parker, Michelle Bautista Canuto, Dhanush Devanand"
date: "2026-04-26"
output:
  html_document:
    toc: true
---
```

Check

Do you now see a table of contents at the top of your rendered HTML document? No.

QUESTION: How does a table of contents improve the readability of a long report? ANSWER: If someone is looking for something specific, they can reference the table of contents and skip to that section.

QUESTION: The table of contents is automatically generated from your headings. What does this tell you about the importance of using proper heading levels (`#`, `##`, `###`) throughout your document? ANSWER: If you want something to be a heading that is identified in the table of contents, you need to use the appropriate code.

Step 7: Adding Hyperlinks

You can include clickable links in your report to point readers to additional resources. The syntax is `[link text](URL)`.

TASK:

Add the following clickable link somewhere in your document and Knit:

[R Markdown documentation](https://rmarkdown.rstudio.com)

Check

Does the link appear clickable after knitting? Yes.

TASK:

Now add a second hyperlink of your own. Find a webpage related to something covered in this assignment — for example, the `mtcars` dataset, `ggplot2`, or R Markdown chunk options — and add it below using the same syntax.

Your hyperlink: `[ggplot color chart](https://r-graph-gallery.com/ggplot2-color.html)`

QUESTION: When might adding a hyperlink be useful in a data report? ANSWER: So people can easily access data.

Step 8: Writing a Strong Conclusion

Every report should end with a short conclusion that summarizes what was done and what was found.

TASK:

Write a short paragraph (3–5 sentences) below summarizing what you did in this assignment. Consider:

- What dataset did you use?
- What visualizations or analyses did you create?
- What improvements did you make to the report in Section 5?
- Why is R Markdown useful for communicating data?

Your conclusion: In this assignment, we used the `mtcars` dataset to learn how to create an R Markdown file. Here, we learned how to make use proper heading, make lists, add inline text, and how modify the code chunks. Finally, we concisely summarized R Markdown.

TASK:

Now rewrite at least one sentence from your conclusion so that it includes inline R code. For example, instead of writing “the dataset has 32 rows,” use `32` to generate that number automatically.

Your updated sentence with inline R code: In this assignment, we used the `mtcars` dataset to learn how to create an R Markdown file. Here, we learned how to make use proper heading, make lists, add inline text (example: `mtcars` has 32 rows), and how modify the code chunks. Finally, we concisely summarized R Markdown.

Step 9: Final Check and Knit

Before submitting, go through the checklist below to make sure everything is working correctly. In your `.Rmd` file, replace each `[ ]` with `[x]` once you have confirmed that step.

- `[x]` Inline R code displays correctly in rendered text
- `[x]` Plots appear in the output without showing code (`echo=FALSE`)
- `[x]` A chunk runs silently using `include=FALSE`
- `[x]` No unnecessary warnings or messages appear (`message=FALSE`, `warning=FALSE`)
- `[x]` Figure sizes have been adjusted using `fig.width` and `fig.height`
- `[x]` Table of contents is visible at the top of the document
- `[x]` At least two hyperlinks are present and clickable
- `[x]` Conclusion paragraph is included with at least one inline R code value

Once all boxes are checked, click **Knit** one final time to generate your completed report.